



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

编译系统原理实验报告

了解我的编译器

林晖鹏 王斯毅

年级：2023 级

专业：计算机科学与技术

指导教师：王刚

2025 年 9 月 29 日

摘要

本实验深入探究了 GCC/LLVM 编译工具链的完整工作流程，系统分析了从源代码到可执行文件的四个关键阶段：预处理、编译、汇编和链接。通过设计包含丰富语言特性的 SysY 程序案例，实现了从高级语言到 LLVM IR 中间表示，再到 RISC-V 汇编代码的完整编译链路。实验采用对比分析方法，研究了不同优化级别（O0-O3）对汇编代码生成的影响，观察了常量传播、死代码消除、循环优化等编译优化技术的实际效果。特别地，探索了 LLVM 的自动并行化能力，验证了 Polly 在高级优化级别下对循环结构的并行化处理。通过手工编写等价的 LLVM IR 和 RISC-V 汇编程序，并与编译器生成结果进行对比分析，揭示了编译器内部工作机制和优化策略。实验结果表明，编译系统通过多层次、模块化的处理流程，能够将高级语言程序高效转化为目标机器代码，同时在不同优化权衡中实现性能与代码质量的平衡。本实验中实验环境为 Docker 中的 24.04 的 Ubuntu，代码地址为<https://github.com/Absurdaaa/Compiler-Principles.git>

关键字：编译系统，LLVM，GCC，SysY，RISC-V，代码优化

目录

一、 引言	1
二、 编译流程研究	1
(一) 编译流程概述	1
(二) 预处理器阶段	2
(三) 编译器阶段	4
(四) 汇编器阶段	5
(五) 链接器阶段	6
(六) 小结	8
(七) 不同优化力度的编译结果对比	8
三、 SysY 到 LLVM IR 与汇编的实现与验证	10
(一) Sysy 语言介绍及案例设计	10
1. SysY 语言介绍	10
2. 案例设计	10
(二) LLVM 程序编写	12
1. 基本结构	12
2. LLVM 等价程序编写与注释说明	12
3. LLVM 等价程序的编译及验证	16
4. LLVM IR 程序的不同优化对比	17
5. LLVM IR 程序自动并行化尝试	20
(三) 汇编程序编写	21
1. 数据段和字符串常量	21
2. 函数设计思路	22
3. 主函数设计思路	24
4. 汇编程序验证	26
5. 汇编代码比较	27

四、 实验总结

29

一、引言

编译系统在现代计算机体系中扮演着至关重要的角色。它不仅是高级语言与底层硬件之间的桥梁，也是程序性能优化与跨平台移植的关键工具。通过编译系统，开发者能够将抽象的高级语言程序逐步转化为可在特定硬件平台上运行的高效机器代码。对于学习与研究编译原理的学生而言，深入理解编译系统的各个阶段，有助于全面把握程序执行的本质过程。

本实验的研究目标主要包括两个方面：首先，探究 **GCC/LLVM** 工具链的完整工作过程，详细分析预处理器、编译器、汇编器与链接器在源代码到可执行文件转化过程中所承担的功能，特别是编译器内部的词法分析、语法分析、语义分析、中间代码生成与优化等关键环节；其次，基于 **SysY** 语言设计并实现一个完整的编译实验流程，将 **SysY** 源程序转化为 **LLVM IR**，进一步编译为 **RISC-V 汇编程序**，并通过汇编与链接步骤生成可运行的目标程序，从而验证编译结果的正确性。

小组分工 本小组选题为 RISC-V，引言部分由林晖鹏同学完成，问题一由小组内同学各自完成，问题二中任务一由林晖鹏同学完成，任务二由王斯毅同学完成，其余部分由两人共同完成。

二、编译流程研究

(一) 编译流程概述

在现代编译系统中，源程序从 **.c** 文件逐步转化为最终的可执行文件，一般需要经过四个主要阶段：**预处理**、**编译**、**汇编**与**链接**。其基本流程如下所示：



图 1: 编译基本流程

下面基于斐波那契数列的基础代码来研究用 **GCC** 编译的具体流程，这份代码中包含了尽可能丰富的 **C** 语言基础特性（宏定义、全局变量、函数、头文件...）：

```
1  #include "sylib.h"
2  #define INIT_A 0
3  #define INIT_B 1
4
5  int i = 1;
6
7  int add(int a, int b){
8      return a + b;
9  }
10
11 //斐波那契数列
12 int main(){
13     int a, b, t, n;
14     a = INIT_A;
15     b = INIT_B;
16     n = getint();
```

```
17     putint(a);
18     putchar('\n');
19     putint(b);
20     putchar('\n');
21     while (i < n){
22         t = b;
23         b = add(a, b);
24         putint(b);
25         putchar('\n');
26         a = t;
27         i = i + 1;
28     }
29 }
```

(二) 预处理器阶段

预处理器阶段在编译的最前端工作，其输入为源文件 `.c/.h`，输出为展开后的中间文件 `.i`（为纯文本的 C 代码）。它**不做语义检查与代码生成**，主要完成如下工作：

1. **文件包含处理**：预处理器会处理 `#include` 指令，将对应的头文件内容直接插入到当前源文件中，相当于“文本替换”。这样，编译器在后续编译阶段可以完整地看到所有函数、类型与变量的声明。例如，`#include "stdlib.h"` 会将该头文件中的全部声明插入到源文件，从而使 `putch` 等函数在编译时被正确识别。
2. **宏定义与宏展开**：预处理器会解析代码中的 `#define` 宏定义，并在代码中出现宏调用的地方进行替换。包括对象宏（如 `#define INIT_A 0`）、函数宏、以及变参宏等。在展开时，预处理器会将调用处替换为具体文本。例如，在斐波那契函数中若使用了 `INIT_A` 和 `INIT_B`，则会被展开为 `0` 与 `1`。
3. **条件编译**：通过 `#if`、`#ifdef`、`#ifndef`、`#else`、`#elif`、`#endif` 等指令，预处理器可以根据条件选择性地包含或排除某些代码片段。这常用于跨平台兼容、调试开关或版本控制。例如，`#ifdef DEBUG` 可以在调试模式下启用调试输出，而在发布版本中自动忽略。
4. **注释删除**：预处理器会在输出的 `.i` 文件中移除所有注释，包括单行注释（`// ...`）与多行注释（`/* ... */`）。这样保证了后续的词法分析器只处理有效的代码符号，不会被注释干扰。
5. **其他指令处理**：除了以上核心功能，预处理器还会处理 `#undef`（取消宏定义）、`#pragma`（向编译器传递特定编译选项）、`#line`（修改文件名和行号信息）等特殊指令。编译器在此阶段完成这些指令的展开或替换，确保生成的预处理结果能够正确指导后续编译阶段。

通过这一系列步骤，预处理器将源文件转化为一个不含宏、注释和未解析条件编译的“展开版”代码文件，为后续的词法分析和语法分析阶段提供了完整、纯净的输入。示例命令：

```
gcc -E main.c -o main.i
```

运行这个指令之后得到超长的 `.i` 文件代码，代码内容一共分为**四部分**，如下面代码中注释所示：

1. 头部 # 指令和头文件展开

这部分先是显示了行号和文件名信息，用于调试和错误定位；然后把所有 #include 的内容都插入进来了，包括标准库和自己的库。

2. 标准库类型和函数声明

这部分写了标准库和系统头文件里的类型声明以及函数声明。

3. sylib.h 的声明和全局变量

这部分是自定义库 sylib.h 的函数声明，比如输入输出、计时等；以及一些全局变量声明

4. 主程序代码

这部分是主程序代码。

```
1 // 1. 头部 # 指令和头文件展开
2 # 0 "main.c"
3 # 0 "<built-in>"
4 # 0 "<command-line>"
5 ...
6 // 2. 标准库类型和函数声明
7 typedef unsigned char __u_char;
8 typedef unsigned short int __u_short;
9 typedef unsigned int __u_int;
10 typedef unsigned long int __u_long;
11 ...
12 // 3. sylib.h 的声明和全局变量
13 # 8 "sylib.h"
14 int getint(), getch(), getarray(int a[]);
15 float getfloat();
16 int getfarray(float a[]);
17 ...
18 // 4. 主程序代码
19 # 2 "main.c" 2
20 int i = 1;
21 int add(int a, int b){
22     return a + b;
23 }
24 int main(){
25     ...
26 }
```

预处理器常用命令与开关 除了上面那个基础的预处理指令，还可以有很多开关：

- -D< 宏名 > 表示增加某个宏定义，如 -DMAX_N=1024。
- -U< 宏名 > 表示取消（undefine）某个宏
- -Iinclude 编译器会优先到 include/ 目录查找用户头文件。
- -C 保留注释（默认会删除）
- -P 去掉 #line 标记

(三) 编译器阶段

在完成预处理之后，编译器接收展开后的 .i 文件，并将其逐步转化为汇编代码 .s 文件。在这一阶段，编译器不仅仅是“翻译”，而是一个复杂的语言处理系统，通常可划分为前端、中端和后端三个部分。具体包括以下几个步骤：

1. **词法分析**：编译器通过词法分析器将源程序的字符流转化为记号 (Token) 序列，如关键字、标识符、常量、运算符等。例如，语句 `int a = 5;` 会被分解为关键字 `int`，标识符 `a`，运算符 `=`，常量 `5`，以及分号 `;`；

可以通过下面这个指令查看词法分析的结果：

```
clang -E -Xclang -dump-tokens main.c 2> tokens.txt
```

2. **语法分析**：编译器使用语法分析器根据文法规则将 Token 序列解析为抽象语法树 (AST)。抽象语法树清晰地反映了程序的层次结构，如函数、条件语句、循环等。

可以通过下面这个指令查看抽象语法树结构：

```
clang -E -Xclang -ast-dump main.c
```

3. **语义分析**：在得到 AST 后，编译器检查语义正确性，包括类型检查、作用域解析、符号表维护等。例如，判断数组下标是否为整数、函数调用参数与声明是否匹配等。
4. **中间表示 (IR) 生成**：语义正确的 AST 会被翻译为编译器内部的中间表示 (IR)。GCC 使用 GIMPLE、RTL 等 IR；LLVM 使用 LLVM IR。会生成一个 .ll 中间代码，这里不在赘述，在下文有详细探索。

示例命令：

```
clang -S -emit-llvm main.c -o main.ll # 生成 LLVM IR
gcc -fdump-tree-all-graph main.c # GCC 中间树 IR
```

5. **优化**：编译器在 IR 上进行多轮优化，如常量折叠、死代码消除、循环优化、寄存器分配等。例如，表达式 `3*4` 在优化阶段会直接替换为常量 `12`。

示例命令：

```
opt -S -O0 main.ll -o main_O0.ll
opt -S -O1 main.ll -o main_O1.ll
opt -S -O2 main.ll -o main_O2.ll
opt -S -O3 main.ll -o main_O3.ll
```

6. **代码生成**：最后，后端将优化后的 IR 翻译为特定体系结构的汇编代码 (如 x86 或 RISC-V)。

示例命令：

```
clang -S main.c -o main_x86.s # 生成 x86 格式目标代码
riscv64-unknown-elf-gcc -S main.c -o main_riscv.s #riscv64 格式
llc main.ll -o main_llvm.s # LLVM 生成目标代码
```

通过以上步骤，编译器完成了从源代码到汇编代码的完整转换过程。其中，前端保证语义正确，中端专注于优化，后端生成面向目标平台的高效汇编，为后续汇编器阶段输入做好准备。

当然，也可以一条指令一步到胃，生成的是平台相关的汇编源代码，这里环境用的 x86，所以为 x86 的汇编代码。

```
gcc -S main.i -o main.s
```

生成下面代码：

```
1  .file 1 "main.c"
2  .text
3  .globl _sysy_start
4  .bss
5  .align 16
6  .type _sysy_start, @object
7  .size _sysy_start, 16
8  _sysy_start:
9  .zero 16
10 .globl _sysy_end
11 .align 16
12 .type _sysy_end, @object
13 .size _sysy_end, 16
14 _sysy_end:
15 .....
```

(四) 汇编器阶段

编译器输出的 .s(汇编源文件)会交给汇编器处理,生成可重定位目标文件 .o(Linux/x86_64 上通常是 ELF Relocatable)。这一阶段的核心工作不是“再理解高级语言”，而是对汇编文本做指令编码、伪指令展开、节与符号管理、重定位项生成等低层操作，为链接器做准备。

汇编器有啥用

1. **解析与伪指令展开**：读取 .s 文本，识别指令与伪指令（如 .text, .data, .bss, .globl, .align, .type, .size, .string, .zero 等），把伪指令转化为节内容和元数据。
2. **指令编码**：将如 addl %edx, %eax 等汇编指令编码为机器指令字节序列，写入 .text 节。
3. **符号表与重定位**：为出现的符号（函数、全局变量、外部引用）建立 .symtab；对尚未确定地址的引用（例如 call add, mov i(%rip), %eax）生成 .rel[a].text 等重定位表项（如 R_X86_64_PC32, R_X86_64_GOTPCREL）。
4. **节与对齐**：把代码/数据分别放入 .text/.data/.bss/.rodata 等节；根据 .align 做对齐填充；.bss 用 .zero 声明未初始化数据。
5. **调试信息与异常表**：如果编译时带 -g 或有 .cfi_* 指令，会产生 .debug_*、.eh_frame 等节，供调试/栈回溯。

示例命令

```
gcc -c main.s -o main.o      # GNU as 在后面被调用
clang -c main.s -o main.o    # Clang 集成汇编器
```


运行上面的指令之后，我们从.s 文件得到一个.o 的二进制代码文件。下面介绍举一个重定位的例子：

小示例：从调用到重定位 假设 main.s 中有一条调用：

```
1      call add          # add 在别的目标文件里定义
```

此时 main.o 里 .text 节对应位置会是一个**未决**的相对跳转，汇编器会在 .rela.text 中生成一个类似：

```
Relocation section '.rela.text' ...
Offset          Info          Type          Sym.Value  Sym.Name
00000000000000xx 00000000y R_X86_64_PLT32 00000000   add+0
```

含义：链接器稍后把 call 的目标修补为符号 add 的真实地址（可能通过 PLT/GOT）。

注意：llvm-as 不是 x86 汇编器 llvm-as 的 as 指 “assemble LLVM IR”，它把 LLVM IR 文本 .ll 组装成位码 .bc，不是把 x86 汇编 .s 变 .o。x86 汇编请用 gcc/clang -c 或 as。

（五） 链接器阶段

链接器接收多个 .o（以及静态库 .a/动态库 .so），生成**可执行文件或共享库**。Linux/x86_64 下常见的链接器有 ld.bfd、gold 和 lld；通常我们通过 gcc/clang 驱动来调用它们，以自动加入启动文件与标准库。

链接器在做什么

1. **符号解析**：把各 .o 与库中的全局符号（函数/变量）对上号。处理强/弱符号、重复定义、未定义引用等冲突。
2. **节合并与地址分配**：把所有输入文件的 .text/.data/.rodata/.bss 等节合并，决定它们在输出文件中的**虚拟地址布局**；生成程序头。
3. **重定位**：遍历每个 .o 的重定位表，把其中的占位（如 call add、mov i(%rip),%eax）修补为最终地址/位移；对动态链接场景生成/填充 PLT（Procedure Linkage Table）和 GOT（Global Offset Table）。
4. **生成可执行/共享库**：写出 ELF 可执行文件（ET_EXEC/ET_DYN），填入入口点（_start），并记录动态装载器路径（.interp 节，如 /lib64/ld-linux-x86-64.so.2）。

常用命令（用编译器驱动最省心）

```
# 1) 最基本：把 .o 链成可执行文件（自动加 C 运行时与 libc）
gcc main.o util.o -o app

# 2) 和库链接
ar rcs libmylib.a util.o          # 先打包一个静态库
gcc main.o -L. -lmylib -o app     # 链接静态库（-L 指定库路径，-l 前缀省略 lib 与后缀）
```

```
# 3) 生成并使用动态库
gcc -fPIC -c util.c -o util.pic.o
gcc -shared -o libmylib.so util.pic.o
gcc main.o -L. -lmylib -Wl,-rpath,'$$ORIGIN' -o app # 运行时在可执行同目录找库

# 4) 纯 ld
ld -o app main.o util.o -lc -dynamic-linker /lib64/ld-linux-x86-64.so.2
```

静态链接 vs 动态链接

- **静态链接 (.a)**: 把库的代码拷贝进可执行文件; 运行时不依赖外部 .so; 体积大, 更新不灵活。可用 `gcc -static` 生成纯静态可执行 (不一定在所有系统可运行)。
- **动态链接 (.so)**: 可执行文件只记下“需要哪个 .so”, 运行时由动态装载器加载; 通过 PLT/GOT 支持延迟绑定 (lazy binding), 体积小、更新方便。

跨文件调用示例 (符号如何被解决)

```
1 // util.c
2 int add(int a, int b) { return a + b; }
3
4 // main.c
5 extern int add(int, int);
6 int main() { return add(2, 3); }
```

```
gcc -c util.c -o util.o
gcc -c main.c -o main.o
readelf -r main.o | sed -n '1,50p' # 可见对 add 的重定位项
gcc main.o util.o -o app # 链接后重定位被修补为真实地址
```

常见链接错误与定位

- **undefined reference to 'foo'**: 缺少定义。检查是否忘了把 `foo.o` 或 `libfoo.a/.so` 放进链接命令; 静态库的**顺序**很重要 (`gcc main.o -lfoo` OK, 反过来可能不行)。
- **multiple definition of X**: 同名全局符号被定义了多次 (例如同时编译了 `sylib.c` 并在别处又定义了同名全局; 或把 `.c` 当成头文件去 `#include`)。只保留一个定义, 其余用 `extern` 声明。
- **cannot find -lfoo**: 找不到库。补上 `-L/path/to/lib` 或设置 `LD_LIBRARY_PATH`、`rpath`。
- **ABI/架构不匹配**: 如把 `x86_64` 可执行去链接 `32` 位库; 或 `RISC-V` 目标与 `x86` 主机混用。用 `file libxxx.so`、`readelf -h` 确认架构与 ABI。

链接器做了哪些“看不见”的事

- **启动与收尾对象**: 通过 `gcc` 链接时会自动加入 `crt1.o/crti.o/crtn.o` 等, 设置入口 `_start` 并在进入 `main` 前完成运行时初始化。

- **PLT/GOT 机制**：对动态库函数调用生成 plt 桩和 got 入口，支持懒绑定与地址无关代码 (PIC)。
- **节合并与优化**：合并同名只读常量 (.rodata)、消除未引用节 (--gc-sections, 需配 -ffunction-sections -fdata-sections)。

(六) 小结

综上所述，从源程序 main.c 到最终可执行文件 a.out，编译系统依次经历了预处理、编译、汇编和链接四个阶段，每一阶段各有侧重点：

1. **预处理器阶段**：主要完成代码级别的“文本替换”，包括头文件展开、宏定义与宏展开、条件编译指令处理，以及注释删除等，输出纯净的 .i 文件，为后续语法分析准备完整输入。
2. **编译器阶段**：将 .i 文件转化为汇编源文件 .s。这一过程内部又可细分为前端（词法分析、语法分析、语义分析）、中端（生成中间表示 IR 并进行多轮优化）、后端（目标相关的指令选择与寄存器分配）。最终输出面向特定体系结构的汇编代码。
3. **汇编器阶段**：将 .s 汇编为目标文件 .o。此时，汇编器负责完成指令编码、节与符号表生成、伪指令处理及重定位信息的记录。输出文件不再是文本，而是符合 ELF 格式的二进制，可供链接器处理。
4. **链接器阶段**：将多个 .o 文件与运行时库（如 SysY 的 sylib.o 或系统的 libc.so）进行符号解析与重定位，合并节表并确定最终内存布局，生成具有入口点 _start 的可执行文件。动态链接场景下，还会生成 GOT/PLT 以支持运行时加载。

通过这一套流程，源代码逐步降低抽象层级，从高层的 C 程序，经由 IR 与汇编语言，最终转化为底层机器码。每个阶段的输出既是下个阶段的输入，也是调试与理解编译器工作原理的重要切入点。这种层次化、流水线式的设计保证了编译过程的模块化和可移植性，也使得我们能够实验中针对不同阶段进行对比与优化分析。

(七) 不同优化力度的编译结果对比

本小节探究不同优化力度下的编译代码区别，运行下面指令：

```
gcc -S -O0 main.c -o main0.s
gcc -S -O1 main.c -o main1.s
gcc -S -O2 main.c -o main2.s
gcc -S -O3 main.c -o main3.s
```

这里使用 diff 做编译代码的对比，可能是这里程序并不是很复杂，这里 O2 和 O3 优化的结果没有区别，所以不再分析。

O1 相比 O0 的优化 O1 优化主要体现在变量分配和函数调用方式上。例如，在 O0 下，add 函数的实现如下：

```
1 pushq %rbp
2 movq %rsp, %rbp
3 movl %edi, -4(%rbp)
```

```

4 movl    %esi, -8(%rbp)
5 movl    -4(%rbp), %edx
6 movl    -8(%rbp), %eax
7 addl    %edx, %eax
8 popq    %rbp

```

而在 O1 下，编译器直接将加法内联为一条指令：

```

1 leal    (%rdi,%rsi), %eax

```

这不仅减少了函数调用的开销，也显著简化了指令数量。

此外，O1 优化后，循环变量和临时变量更多地分配在寄存器（如

O2 相比 O1 的优化 O2 优化进一步提升了代码效率，主要体现在指令重排、对齐和冗余消除。例如，O2 下出现了如下指令：

```

1 .p2align 4
2 .section .text.startup,"ax",@progbits
3 xorl    %eax, %eax
4 xorl    %edi, %edi
5 xorl    %edx, %edx

```

这些 xorl 指令用于快速清零寄存器，替代了多次 movl \$0, ... 的写法，减少了指令数。代码段也增加了 .p2align 和 .section，有助于提升指令对齐和启动效率。

循环和分支结构在 O2 下进一步简化，分支跳转更紧凑，减少了冗余跳转。例如，O2 下循环体采用了 .p2align 优化，并消除了多余的 movl 指令，整体结构更有利于 CPU 执行。

表 1: 不同优化级别下编译代码主要差异

优化点	O0	O1	O2
局部变量分配	栈上分配, 频繁 movl 访问	寄存器分配, 减少内存访问	寄存器分配, 进一步减少冗余
函数调用	完整调用, 参数入栈	简单函数内联, 减少调用	内联, 进一步消除冗余调用
指令数量	多, 结构接近源码	更少, 结构紧凑	最少, 结构高度优化
循环与分支	跳转多, 结构原始	跳转简洁, 循环优化	跳转最简, 分支高效
初始化	多次 movl 清零	xorl 清零, 减少指令	xorl 清零, 彻底消除冗余
代码段与对齐	普通 .text 段	增加 .p2align 优化对齐	增加 .section、.p2align, 启动段优化

三、 SysY 到 LLVM IR 与汇编的实现与验证

(一) Sysy 语言介绍及案例设计

1. SysY 语言介绍

SysY 语言是编译原理课程实验中广泛使用的一种简化版 C 语言，其设计目标在于既保持 C 语言的核心特性，又降低实现复杂度，从而方便理解和实现编译器前端与后端的基本功能。SysY 的语法和语义与 ANSI C 有较高相似度，同时在功能上做了适当裁剪与扩展。其主要特性如下：

- **基本数据类型：**仅包含整型 `int`、浮点型 `float` 以及布尔型 `bool`，去除了指针等复杂类型；
- **运算与表达式：**支持四则运算、取余、逻辑运算、关系运算以及复合表达式；
- **变量与常量：**支持局部变量与全局变量定义，支持常量声明；
- **语句结构：**包括顺序语句、赋值语句、条件分支 (`if-else`)、循环结构 (`while`, `for`)、`return` 语句等；
- **函数：**支持函数定义与调用，包括递归调用，但不支持可变参数；
- **数组：**支持一维与多维数组的定义与访问；
- **输入输出：**通过配套的 SysY 运行时库提供标准输入输出接口（如 `getint()`、`putint()` 等）。

作为教学语言，SysY 既能覆盖编译器设计中最核心的语法分析、语义分析和中间代码生成等环节，又避免了完整 C 语言中复杂特性的干扰。因此，它成为了编译系统原理课程中进行实验与项目实践的理想平台。

2. 案例设计

下面我们设计一个基本涵盖全部编译器要支持的语言特性的 SysY 代码案例，代码如下，特性均在注释中有说明：

```
1 // 函数声明
2 int add(int a, int b);
3 int factorial(int n);
4 int fibonacci(int n);
5
6 void putf(char a[], ...);
7
8 // 主函数
9 int main()
10 {
11     int a, b, sum, fact, fib; // 变量声明（整型变量）
12
13     // 赋值与运算
14     a = 5;
15     b = 10;
```

```
16     sum = add(a, b); // 函数调用
17
18     printf("Sum of %d and %d is %d\n", a, b, sum); //SysY 拓展支持内建 I/O 函数 printf
19
20     fact = factorial(5); // 函数递归调用
21     printf("Factorial of 5 is %d\n", fact);
22
23     fib = fibonacci(6);
24     printf("Fibonacci of 6 is %d\n", fib);
25
26     // 条件分支
27     if (fact > 100) printf("The factorial is large\n");
28     else printf("The factorial is small\n");
29
30     // 循环
31     for (int i = 0; i < 5; i++) printf("Loop iteration %d\n", i);
32
33     // 数组操作
34     int arr[5] = {1, 2, 3, 4, 5}; // 数组定义和初始化
35     printf("Array element at index 2: %d\n", arr[2]);
36
37     return 0; //return 语句
38 }
39
40 // 加法函数
41 int add(int a, int b)
42 {
43     return a + b;
44 }
45
46 // 阶乘函数
47 int factorial(int n)
48 {
49     if (n <= 1) return 1;
50     else return n * factorial(n - 1); // 递归调用
51 }
52
53 // 斐波那契函数
54 int fibonacci(int n)
55 {
56     if (n <= 1) return n;
57     else return fibonacci(n - 1) + fibonacci(n - 2);
58 }
```

编写完示例程序后，我们在终端中输入指令

```
1 clang main.c ../../lib/sylib.c -o main
2 ./main
```

得到如图2结果, 需要注意的是, 在 Sysy 链接库中, 存在定义的析构函数, 在结束周期的时候会返回总时间, 所以最后有一个 **TOTAL:0H-0M-0S-0us**。

```
• docay@LAPTOP-IKL4J1DU:~/Compiler-Principles/lab1/t2/LLVM-IR$ clang main.c ../../lib/sylib.c -o main
• docay@LAPTOP-IKL4J1DU:~/Compiler-Principles/lab1/t2/LLVM-IR$ ./main
Sum of 5 and 10 is 15
Factorial of 5 is 120
Fibonacci of 6 is 8
The factorial is large
Loop iteration 0
Loop iteration 1
Loop iteration 2
Loop iteration 3
Loop iteration 4
Array element at index 2: 3
TOTAL: 0H-0M-0S-0us
```

图 2: 示例代码运行结果

(二) LLVM 程序编写

LLVM 中间表示 (LLVM IR) 是一种面向静态单赋值形式的低级中间语言, 它既保留了高级语言的抽象特性, 又接近汇编语言的控制粒度, 因此非常适合进行编译器优化和跨平台代码生成。在本实验中, 我们通过手动编写 LLVM IR 程序, 使其与 SysY 源程序在语义上保持等价, 并能够进一步生成 RISC-V 汇编代码。

1. 基本结构

LLVM IR 程序由以下几个部分构成:

模块 (Module): 代表整个程序, 是函数和全局变量的集合;

函数 (Function): 使用 `define` 关键字定义, 包含多个基本块;

基本块 (Basic Block): 以标签标识, 包含一系列顺序执行的指令, 必须以控制流指令结束 (如 `br` 或 `ret`);

指令 (Instruction): 包括算术运算 (`add`, `mul`)、比较 (`icmp`)、内存访问 (`load`, `store`)、函数调用 (`call`) 等。

2. LLVM 等价程序编写与注释说明

为了更好地理解 LLVM IR 的语法结构与语义, 本实验编写了一个等价于 SysY 程序的 LLVM IR 文件。代码中加入了详细注释, 以解释每一类语句的功能与写法。

```
1 ; ModuleID = 'main.c'
2 source_filename = "main.c"
```

```

3 target datalayout = "e-m:e-p:64:64-i64:64-n32:64-S128"
4 target triple = "riscv64-unknown-elf"
5
6 ; -----
7 ; 全局常量区
8 ; -----
9 ; LLVM IR 中的全局字符串常量以 @ 开头, 使用 constant 定义,
10 ; unnamed_addr 表示地址不唯一, align 指定对齐方式。
11 @.str = private unnamed_addr constant [24 x i8] c"Sum of %d and %d is %d\0A\00", align 1
12 @.str.1 = private unnamed_addr constant [22 x i8] c"Factorial of 5 is %d\0A\00", align 1
13 @.str.2 = private unnamed_addr constant [22 x i8] c"Fibonacci of 6 is %d\0A\00", align 1
14 @.str.3 = private unnamed_addr constant [24 x i8] c"The factorial is large\0A\00", align
    ↪ 1
15 @.str.4 = private unnamed_addr constant [24 x i8] c"The factorial is small\0A\00", align
    ↪ 1
16 @.str.5 = private unnamed_addr constant [19 x i8] c"Loop iteration %d\0A\00", align 1
17 @.str.6 = private unnamed_addr constant [30 x i8] c"Array element at index 2: %d\0A\00",
    ↪ align 1
18
19 ; -----
20 ; 外部函数声明
21 ; -----
22 ; 声明 sylib 库中的 putf 函数, 返回类型为 void, 参数为不定长 (...).
23 declare void @putf(ptr, ...)
24
25 ; -----
26 ; 函数 add
27 ; -----
28 ; 函数定义以 define 开头, dso_local 表示默认符号可见性。
29 define dso_local i32 @add(i32 %a, i32 %b) {
30 entry:
31     %sum = add nsw i32 %a, %b    ; 执行加法运算, nsw 表示有符号溢出未定义
32     ret i32 %sum                ; 返回结果
33 }
34
35 ; -----
36 ; 函数 factorial (递归实现)
37 ; -----
38 define dso_local i32 @factorial(i32 %n) {
39 entry:
40     %cmp = icmp sle i32 %n, 1    ; if (n <= 1)
41     br i1 %cmp, label %ret1, label %rec
42 ret1:                                ; 基本块 ret1
43     ret i32 1
44 rec:                                ; 基本块 rec
45     %sub = sub nsw i32 %n, 1
46     %call = call i32 @factorial(i32 %sub)
47     %mul = mul nsw i32 %n, %call
48     ret i32 %mul

```



```

49 }
50
51 ; -----
52 ; 函数 fibonacci (递归实现)
53 ; -----
54 define dso_local i32 @fibonacci(i32 %n) {
55     entry:
56     %cmp = icmp sle i32 %n, 1
57     br i1 %cmp, label %ret, label %rec
58     ret:
59     ret i32 %n
60     rec:
61     %sub1 = sub nsw i32 %n, 1
62     %call1 = call i32 @fibonacci(i32 %sub1)
63     %sub2 = sub nsw i32 %n, 2
64     %call2 = call i32 @fibonacci(i32 %sub2)
65     %sum = add nsw i32 %call1, %call2
66     ret i32 %sum
67 }
68
69 ; -----
70 ; 主函数 main
71 ; -----
72 define dso_local i32 @main() {
73     entry:
74     ; -----
75     ; 局部变量分配: alloca 指令在栈上分配内存
76     ; 返回的指针类型是 i32
77     ; -----
78     %a = alloca i32, align 4 ; int a;
79     %b = alloca i32, align 4 ; int b;
80     %sum = alloca i32, align 4 ; int sum;
81     %fact = alloca i32, align 4 ; int fact;
82     %fib = alloca i32, align 4 ; int fib;
83     %i = alloca i32, align 4 ; int i;
84     %arr = alloca [5 x i32], align 16; int arr[5];
85
86     ; 初始化变量: store 指令写入内存
87     store i32 5, ptr %a, align 4 ; a = 5;
88     store i32 10, ptr %b, align 4 ; b = 10;
89
90     ; 调用 add 函数并存储返回值
91     %0 = load i32, ptr %a, align 4 ; 取出 a
92     %1 = load i32, ptr %b, align 4 ; 取出 b
93     %call = call i32 @add(i32 %0, i32 %1) ; 调用 add(a, b)
94     store i32 %call, ptr %sum, align 4 ; sum = add(a, b)
95
96     ; 打印加法结果
97     %2 = load i32, ptr %a, align 4

```

```

98     %3 = load i32, ptr %b, align 4
99     %4 = load i32, ptr %sum, align 4
100    call i32 (ptr, ...) @putf(ptr @.str, i32 %2, i32 %3, i32 %4)
101
102    ; 调用 factorial(5)
103    %call1 = call i32 @factorial(i32 5)
104    store i32 %call1, i32* %fact, align 4
105    %fact_val = load i32, i32* %fact, align 4
106    call i32 (ptr, ...) @putf(ptr @.str.1, i32 %fact_val)
107
108    ; 调用 fibonacci(6)
109    %call2 = call i32 @fibonacci(i32 6)
110    store i32 %call2, i32* %fib, align 4
111    %fib_val = load i32, i32* %fib, align 4
112    call i32 (ptr, ...) @putf(ptr @.str.2, i32 %fib_val)
113
114    ; -----
115    ; 条件分支 if (fact > 100)
116    ; icmp: 比较两个整数
117    ; br i1: 根据布尔值跳转到不同的基本块
118    ; -----
119    %fact_cmp_val = load i32, i32* %fact, align 4
120    %cmp = icmp sgt i32 %fact_cmp_val, 100 ; fact > 100 ?
121    br i1 %cmp, label %large, label %small ; 若真则跳 large, 否则跳 small
122
123    large:                                ; 大分支
124        call i32 (ptr, ...) @putf(ptr @.str.3)
125        br label %endif
126
127    small:                                ; 小分支
128        call i32 (ptr, ...) @putf(ptr @.str.4)
129        br label %endif
130
131    endif:                                ; 两个分支合流的基本块
132
133    ; -----
134    ; 循环 for (i = 0; i < 5; i++)
135    ; LLVM IR 将 for 转换为循环条件块 loop.cond
136    ; 与循环体 loop.body, 再到 loop.end
137    ; -----
138    store i32 0, ptr %i, align 4          ; i = 0
139    br label %loop.cond                  ; 跳转到循环条件检查
140
141    loop.cond:                            ; 循环条件块
142        %loop_i = load i32, ptr %i, align 4
143        %cmp1 = icmp slt i32 %loop_i, 5    ; i < 5 ?
144        br i1 %cmp1, label %loop.body, label %loop.end
145
146    loop.body:                            ; 循环体

```

```

147  %loop_i_val = load i32, ptr %i, align 4
148  call i32 (ptr, ...) @putf(ptr @.str.5, i32 %loop_i_val)
149  %loop_i_inc = load i32, ptr %i, align 4
150  %inc = add nsw i32 %loop_i_inc, 1      ; i++
151  store i32 %inc, ptr %i, align 4
152  br label %loop.cond                  ; 跳回条件检查
153
154  loop.end:                            ; 循环结束
155
156  ; -----
157  ; 数组 arr[5] = {1,2,3,4,5}
158  ; getelementptr (GEP) 指令计算数组元素地址
159  ; -----
160  %arrptr = getelementptr inbounds [5 x i32], ptr %arr, i32 0, i32 0
161  store i32 1, ptr %arrptr, align 4
162  %arrptr1 = getelementptr inbounds [5 x i32], ptr %arr, i32 0, i32 1
163  store i32 2, ptr %arrptr1, align 4
164  %arrptr2 = getelementptr inbounds [5 x i32], ptr %arr, i32 0, i32 2
165  store i32 3, ptr %arrptr2, align 4
166  %arrptr3 = getelementptr inbounds [5 x i32], ptr %arr, i32 0, i32 3
167  store i32 4, ptr %arrptr3, align 4
168  %arrptr4 = getelementptr inbounds [5 x i32], ptr %arr, i32 0, i32 4
169  store i32 5, ptr %arrptr4, align 4
170
171  ; 打印 arr[2]
172  %arr2 = load i32, ptr %arrptr2, align 4
173  call i32 (ptr, ...) @putf(ptr @.str.6, i32 %arr2)
174
175  ret i32 0
176 }

```

从上述示例可以看到，LLVM IR 中的语法规则与汇编语言类似，但具有严格的类型系统和 SSA（静态单赋值）特性。变量以 % 前缀表示临时寄存器，全局符号以 @ 前缀表示。控制流通过基本块（label）和分支指令（br）进行组织，而函数调用、算术指令与条件判断的语法形式则与 C 语言语义一一对应。

在编写指令的过程中，发现临时寄存器如果用数字来命名，必须按照已出现的个数 +1 作为编号，否则会报错。另外

通过以上步骤，可以确保手动编写的 LLVM IR 程序正确无误，并为后续汇编与链接实验打下基础。

3. LLVM 等价程序的编译及验证

下面我们来尝试编译并运行这个.ll 文件，结合课程资料中的 sylib 链接库，我们尝试用三种方法去编译并验证这个程序：

用 clang 编译为可执行文件 使用 clang 可以直接将 LLVM IR 文件编译为本地可执行文件。这个指令可以把 LLVM IR 程序 main.ll 与 SysY 的 x86 运行库静态链接，生成可执行文件 main，从而可以在本地主机上直接运行 SysY 程序。

```
1 clang -o main main.ll ../../lib/libsysy_x86.a
```

但是由于我们在编写 LLVM IR 程序的时候，用到了 **OpaquePointersd** 的特性，但是 clang14 版本过低并不支持，无法识别代码程序中的 ptr，一直报错。

使用 LLVM JIT 运行 或者我们可以使用 LLVM 的解释执行器 lli，在本地主机上直接运行 main.ll，并动态加载 SysY 提供的运行库 sylib.so。

```
1 lli -opaque-pointers -load=../../lib/sylib.so main.ll
```

由于我们使用到了 **OpaquePointersd** 的特性，所以这里要加 **-opaque-pointers** 的开关。

基于 LLVM 的交叉编译 再或者，我们可以套用 SysY → LLVM IR → RISC-V 可执行文件 → QEMU 运行的完整链路，从 LLVM IR 继续出发，先把 LLVM IR 转成 RISC-V 汇编，再汇编成目标文件，最后和 SysY 运行库链接，得到可执行文件。

```
1 # 1. LLVM IR 转字节码
2 llvm-as -opaque-pointers main.ll -o main.bc
3
4 # 2. 生成 RISC-V 汇编代码（带硬浮点扩展，推荐与 sylib.o 保持一致）
5 llc -opaque-pointers -march=riscv64 -mattr=+d -filetype=asm main.bc -o
  main_riscv.s
6
7 # 编译 sylib.c 为 RISC-V 目标文件（硬浮点，lp64d ABI）
8 riscv64-unknown-elf-gcc -c ../../lib/sylib.c -march=rv64gc -mabi=lp64d -o
  ../../lib/sylib.o
9
10 # 3. 汇编生成目标文件（ABI和扩展需与 sylib.o 保持一致）
11 riscv64-unknown-elf-as -march=rv64gc -mabi=lp64d main_riscv.s -o main_riscv.o
12
13 # 4. 链接 SysY 运行库（sylib.o），生成最终可执行文件
14 riscv64-unknown-elf-gcc main_riscv.o ../../lib/sylib.o -o main_riscv
15
16 # 5. QEMU 运行
17 qemu-riscv64 main_riscv
```

这里有两点需要注意：第一，程序前后、链接库的浮点类型需要一致，否则会报错不支持；第二，这里在汇编之前同样要加 **-opaque-pointers** 的开关。

小结 由于使用了 opaque-pointers 的原因，只有方法二和方法三能成功运行，可以看到和原程序结果一致。

```
1 Sum of 5 and 10 is 15
2 Factorial of 5 is 120
3 Fibonacci of 6 is 8
4 The factorial is large
5 Loop iteration 0
6 Loop iteration 1
7 Loop iteration 2
8 Loop iteration 3
9 Loop iteration 4
10 Array element at index 2: 3
11 TOTAL: 0H-0M-0S-0us
```

4. LLVM IR 程序的不同优化对比

和用 gcc 编译 c 文件代码一样，编译 LLVM 这个中间代码照样可以开启不同的优化级别。

O0 优化对比 我们使用如下指令，对我们编写的 LLVM 代码进行编译。

```
opt -opaque-pointers -S -O0 main.ll -o main_O0.ll
```

然后用 diff 指令分析差异，可以看到基本上没有什么优化，只有三个地方不同：

1. ModuleID 不同

如果是编译生成，ModuleID 取决于编译的来源文件。

2. 空行及注释被删除

实际运行不用到。

3. 某些变量编号被自动编号

某些变量编号变成了%5、%6 这样自动编号，这通常是 IR 优化或自动生成时的结果。

```
1 < ; ModuleID = 'main.c'
2 > ; ModuleID = 'main.ll'
3 ...
4 < call i32 @putf(ptr @.str, i32 %2, i32 %3, i32 %4) ; putf("Sum of %d and
   ↳ %d is %d\n", a, b, sum);
5 > %5 = call i32 @putf(ptr @.str, i32 %2, i32 %3, i32 %4)
6 ...
7
```

O0 与 O1 优化对比 我们使用如下指令，继续进行 O1 优化探究：

```
opt -opaque-pointers -S -O1 main.ll -o main_O1.ll # O1 基础优化
```

然后用 diff 指令分析与 O0 的差异，可以看到 O1 优化后代码发生了如下变化：

1. 函数属性增强

O1 优化会为函数声明添加更多属性（如 mustprogress、nofree、norecurse、nosync、nounwind、readnone、willreturn 等），以便后续优化和分析。

2. 局部变量和常量折叠

O1 优化会将一些局部变量、临时变量和常量直接折叠到指令中，减少冗余的 load/store 操作。例如循环变量、数组初始化等会直接展开为常量。

3. 控制流简化

O1 优化会将部分分支、循环和条件语句简化为更直接的 select、phi 等 SSA 表达式，减少不必要的跳转和分支。

4. 指令顺序和参数调整

某些算术指令的参数顺序可能会被调整（如 add nsw i32 %b, %a），以便更好地利用寄存器或优化数据流。

5. 属性和元数据增加

O1 优化会为函数和指令增加更多属性和元数据，便于后续编译器分析和优化。

```

1 < declare void @putf(ptr, ...)
2 > declare void @putf(ptr, ...) local_unnamed_addr
3
4 < define dso_local i32 @add(i32 %a, i32 %b) {
5 > ; Function Attrs: mustprogress norecurse nosync nounwind readnone willreturn
6 > define dso_local i32 @add(i32 %a, i32 %b) local_unnamed_addr #0 {
7
8 <   %sum = add nsw i32 %a, %b
9 >   %sum = add nsw i32 %b, %a
10
11 <   %cmp = icmp sle i32 %n, 1
12 >   %cmp = icmp slt i32 %n, 2
13
14 <   ret i32 1
15 >   %common.ret.op = phi i32 [ %mul, %rec ], [ 1, %entry ]
16 >   ret i32 %common.ret.op
17
18 <   %sub = sub nsw i32 %n, 1
19 >   %sub = add nsw i32 %n, -1
20
21 ...

```

可以看出，O1 优化不仅去除了冗余代码，还增强了函数属性、简化了控制流，并对变量和指令做了进一步优化。

O2 与 O1 优化对比 我们使用如下指令，对 LLVM IR 代码进行 O1 和 O2 级别的优化：

```
opt -opaque-pointers -S -O2 main.ll -o main_O2.ll # 标准优化
```

然后用 diff 指令分析差异，可以看到 O2 优化后代码在 O1 基础上进一步发生了如下变化：

1. **常量传播与折叠** O2 优化会将可以确定的变量直接替换为常量。例如，factorial(5) 的结果直接变为 120，减少了运行时计算。
2. **循环与递归优化** O2 优化对递归函数（如 factorial 和 fibonacci）进行了更激进的优化，使用了更复杂的 phi 节点和累加器变量，部分递归被展开或简化为迭代形式。
3. **tail call 优化** O2 优化将部分函数调用变为 tail call（尾递归调用），提升了性能和可优化性。
4. **控制流进一步简化** O2 优化将部分分支和循环结构进一步简化，减少了冗余的跳转和分支，代码结构更加紧凑。
5. **冗余变量和指令消除** O2 优化会消除未使用的字符串常量、变量和冗余指令，使 IR 更加精简。
6. **指令和变量重命名** O2 优化会对部分变量和指令进行重命名和重新编号，便于 SSA 管理和后续优化。

```

1 < %call1 = call i32 @factorial(i32 5)
2 < %1 = call i32 (ptr, ...) @putf(ptr nonnull @.str.1, i32 %call1)
3 ---
4 > %1 = tail call i32 (ptr, ...) @putf(ptr nonnull @.str.1, i32 120)
5
6 < %call2 = call i32 @fibonacci(i32 6)
7 < %2 = call i32 (ptr, ...) @putf(ptr nonnull @.str.2, i32 %call2)
8 ---
9 > %call2 = tail call i32 @fibonacci(i32 6)
10 > %2 = tail call i32 (ptr, ...) @putf(ptr nonnull @.str.2, i32 %call2)
11
12 < %3 = call i32 (ptr, ...) @putf(ptr nonnull %.str.3..str.4)
13 ---
14 > %3 = tail call i32 (ptr, ...) @putf(ptr nonnull @.str.3)
15 ...

```

可以看出，O2 优化在 O1 基础上进一步提升了代码效率，减少了运行时计算，简化了控制流，并进行了更多的常量传播和尾递归优化。

小结 通过以上对比可以看出，LLVM 在不同优化级别下对 IR 的处理差异明显。我们总结如下表??:

表 2: 不同优化级别下 LLVM IR 的对比总结

优化级别	主要特征	典型优化
O0	几乎无优化，接近源代码翻译	保留 alloca/store/load, 仅自动编号和清理注释
O1	基础优化	mem2reg 寄存器提升, 常量折叠, 控制流简化, 添加函数属性
O2	标准优化, 对比 O1 更激进	常量传播, 冗余消除, 循环/递归优化, 尾递归优化 (tail call)

5. LLVM IR 程序自动并行化尝试

我们可以打开一些编译开关进行自动化并行，-polly 会做循环优化，而-polly-parallel 会尝试自动插入 OpenMP 并行化。

```

opt -opaque-pointers -O0 -polly -polly-parallel -S main.ll -o main_parallel_O0.ll
opt -opaque-pointers -O1 -polly -polly-parallel -S main.ll -o main_parallel_O1.ll
opt -opaque-pointers -O2 -polly -polly-parallel -S main.ll -o main_parallel_O2.ll
opt -opaque-pointers -O3 -polly -polly-parallel -S main.ll -o main_parallel_O3.ll

```

但是在 diff 实践中发现，开启 O0O1 优化的时候，并不会对程序进行并行优化，只有开启 O2O3 的时候才会有 Polly 自动并行优化。这是因为高级优化会对循环和递归结构进行归纳变量

分析、控制流简化和内存访问模式分析，为 Polly 提供可分析的循环结构。低级优化（O0/O1）下，代码结构较为原始，Polly 无法识别和处理并行化。

在 LLVM 中，Polly 自动并行优化（`-polly -polly-parallel`）依赖于高级优化级别（O2/O3）对代码结构的预处理。通过对比 O2 和 O3 优化下的并行 IR 结果，可以发现如下特点：

1. 基本块拆分

并行优化会将原有的入口块（entry）拆分为 `entry.split`，以便插入并行调度和依赖分析相关代码。

2. phi 节点来源调整

所有递归和循环相关的 phi 节点，其来源由 `entry` 变为 `entry.split`，确保数据流在并行环境下正确传递。

3. 控制流前驱变化

递归和循环块的前驱由原始入口块变为拆分后的入口块，便于后续插入并行同步或任务分发逻辑。

4. 指令顺序微调

某些算术指令（如 `mul`）的参数顺序可能发生变化，但不影响实际计算结果。

5. O2 与 O3 的区别

由于本部分的案例代码可能过于简单了，这里 O2/O3 优化没有任何区别。但这里 O3 的特性仍待挖掘

```

1 < entry:
2 > entry.split:
3 ...
4 < %accumulator.tr.lcssa = phi i32 [ 1, %entry ], [ %mul, %rec ]
5 > %accumulator.tr.lcssa = phi i32 [ 1, %entry.split ], [ %mul, %rec ]
6 ...
7 < %mul = mul nsw i32 %n.tr3, %accumulator.tr2
8 > %mul = mul nsw i32 %accumulator.tr2, %n.tr3
9 ...

```

补充：O2 与 O3 优化的区别 而查阅资料发现，O2/O3 之间的优化亦有差别：O2 优化包含常规的代码简化、循环优化和部分内联，兼顾编译速度和执行效率。O3 优化则在 O2 基础上进一步激进，包含更大范围的内联、循环展开和向量化，追求极致性能。实际对比发现，O3 优化后 IR 代码结构更紧凑，循环和递归更容易被 Polly 并行化。

（三）汇编程序编写

1. 数据段和字符串常量

在汇编程序中，我们需要将程序中会用到的字符串常量单独存放在数据段中，以便在运行时通过地址引用使用。这些字符串主要用于 `printf` 调用，包含输出提示和变量值的信息。在本程序中，共有 7 个字符串常量，分别对应加法、阶乘、斐波那契、条件分支提示、循环输出以及数组元素输出。我们使用 `.align 3` 指令将它们按 8 字节对齐，这样可以提高访问效率。在后续函数中，通过 `la a0, .str` 加载字符串地址到寄存器，作为 `printf` 的参数传入。

```
1 .section .rodata
2 .align 3
3 .str:
4     .string "Sum of %d and %d is %d\n"
5 .str.1:
6     .string "Factorial of 5 is %d\n"
7 .str.2:
8     .string "Fibonacci of 6 is %d\n"
9 .str.3:
10    .string "The factorial is large\n"
11 .str.4:
12    .string "The factorial is small\n"
13 .str.5:
14    .string "Loop iteration %d\n"
15 .str.6:
16    .string "Array element at index 2: %d\n"
```

2. 函数设计思路

我们的源代码中涉及到的函数有三个，分别是加法函数、阶乘函数以及斐波那契数列函数，下面我们分别介绍每个函数的编写思路。

加法函数

源代码中的加法函数接受两个寄存器的值，直接将两个寄存器中的值相加并将最终结果保存到 a0 寄存器中，这个过程无需栈帧，所以我们可以编写出下面的加法函数的汇编代码：

```
1 # 函数: add(int a, int b)
2 add:
3     add    a0, a0, a1    # 求和 = a + b
4     ret                    # 返回求和结果
```

阶乘函数

阶乘函数是一个递归函数，如果输入的参数 n 小于等于 1，则返回 1，否则返回 $n * \text{factorial}(n-1)$ 。在汇编实现中需要分配栈帧保存返回地址 ra 和 s 寄存器（这里用 s0 保存 n ），防止递归调用覆盖寄存器的值。条件判断通过 ble 指令实现，递归调用通过 call 指令实现，最终结果存放在 a0 寄存器中。按照上述思路我们可以编写出下面的阶乘函数的汇编代码：

```
1 # 函数: factorial(int n)
2 factorial:
3     addi    sp, sp, -16    # 分配栈帧
4     sd      ra, 8(sp)      # 保存返回地址
5     sd      s0, 0(sp)      # 保存 s0
6
7     mv      s0, a0         # s0 = n
8     li      t0, 1          # t0 = 1
9     ble     s0, t0, .Lfact_ret1 # 如果 (n <= 1) 跳转到 ret1
```

```

10
11     # 递归情况:  $n * factorial(n-1)$ 
12     addi    a0, s0, -1      #  $a0 = n - 1$ 
13     call    factorial      #  $a0 = factorial(n - 1)$ 
14     mul     a0, s0, a0      #  $a0 = n * factorial(n - 1)$ 
15
16     ld      ra, 8(sp)       # 恢复返回地址
17     ld      s0, 0(sp)       # 恢复  $s0$ 
18     addi    sp, sp, 16      # 释放栈帧
19     ret
20
21 .Lfact_ret1:
22     li      a0, 1           # 返回 1
23     ld      ra, 8(sp)
24     ld      s0, 0(sp)
25     addi    sp, sp, 16
26     ret

```

斐波那契数列函数

斐波那契函数同样是递归函数，其逻辑是：当 n 小于等于 1 时直接返回 n ，否则返回 $fibonacci(n-1)+fibonacci(n-2)$ 。由于需要保存 n 以及中间计算的 $fibonacci(n-1)$ 的值，栈帧较阶乘函数更大，使用 $s0$ 和 $s1$ 保存中间值。函数通过 `call` 指令进行递归调用，条件分支通过 `ble` 指令实现。返回值最终存放在 $a0$ 寄存器中，调用结束后恢复所有保存寄存器并释放栈帧。

```

1  # 函数: fibonacci(int n)
2  fibonacci:
3      addi    sp, sp, -32
4      sd      ra, 24(sp)
5      sd      s0, 16(sp)
6      sd      s1, 8(sp)
7
8      mv      s0, a0         #  $s0 = n$ 
9      li      t0, 1          #  $t0 = 1$ 
10     ble     s0, t0, .Lfib_ret # 如果 ( $n \leq 1$ ) 跳转到 ret
11
12     # 递归情况:  $fibonacci(n-1) + fibonacci(n-2)$ 
13     addi    a0, s0, -1      #  $a0 = n - 1$ 
14     call    fibonacci      #  $a0 = fibonacci(n - 1)$ 
15     mv      s1, a0         #  $s1 = fibonacci(n - 1)$ 
16
17     addi    a0, s0, -2      #  $a0 = n - 2$ 
18     call    fibonacci      #  $a0 = fibonacci(n - 2)$ 
19
20     add     a0, s1, a0      #  $a0 = fibonacci(n-1) + fibonacci(n-2)$ 
21
22     ld      ra, 24(sp)     # 恢复返回地址
23     ld      s0, 16(sp)     # 恢复  $s0$ 
24     ld      s1, 8(sp)      # 恢复  $s1$ 

```

```

25     addi    sp, sp, 32      # 释放栈帧
26     ret
27
28 .Lfib_ret:
29     mv      a0, s0          # 返回 n
30     ld      ra, 24(sp)
31     ld      s0, 16(sp)
32     ld      s1, 8(sp)
33     addi    sp, sp, 32
34     ret

```

3. 主函数设计思路

主函数是程序的入口，逻辑上包括变量声明、初始化、函数调用、条件分支、循环以及数组操作。首先为局部变量分配栈空间，并保存必要寄存器。随后依次完成以下操作：

1. 初始化变量 a 和 b，并调用加法函数计算 sum，同时通过 printf 输出。
2. 调用阶乘函数计算 fact 并输出结果。
3. 调用斐波那契函数计算 fib 并输出结果。
4. 判断 fact 是否大于 100，通过条件跳转输出相应提示信息。
5. 通过循环结构打印 0 到 4 的迭代信息，每次循环更新循环变量 i。
6. 初始化数组 arr 并打印第三个元素的值。
7. 最终返回 0，恢复保存的寄存器并释放栈空间。

按照上述流程我们可以编写出如下汇编代码：

```

1  # 函数: main()
2  main:
3      addi    sp, sp, -80
4      sd      ra, 72(sp)
5      sd      s0, 64(sp)
6      sd      s1, 56(sp)
7      sd      s2, 48(sp)
8
9      # 初始化 a = 5, b = 10
10     li      t0, 5
11     sw      t0, 0(sp)      # a = 5
12     li      t0, 10
13     sw      t0, 4(sp)      # b = 10
14
15     lw      a0, 0(sp)      # a0 = a
16     lw      a1, 4(sp)      # a1 = b
17     call    add            # a0 = add(a, b)
18     sw      a0, 8(sp)      # sum = a0
19

```

```

20     # printf("Sum of %d and %d is %d\n", a, b, sum)
21     la      a0, .str      # 加载格式字符串地址
22     lw      a1, 0(sp)     # a1 = a
23     lw      a2, 4(sp)     # a2 = b
24     lw      a3, 8(sp)     # a3 = sum
25     call    printf
26
27     li      a0, 5
28     call    factorial     # a0 = factorial(5)
29     sw      a0, 12(sp)    # fact = a0
30
31     la      a0, .str.1
32     lw      a1, 12(sp)    # a1 = fact
33     call    printf
34
35     li      a0, 6
36     call    fibonacci     # a0 = fibonacci(6)
37     sw      a0, 16(sp)    # fib = a0
38
39     la      a0, .str.2
40     lw      a1, 16(sp)    # a1 = fib
41     call    printf
42
43     # if (fact > 100)
44     lw      t0, 12(sp)    # t0 = fact
45     li      t1, 100       # t1 = 100
46     bgt     t0, t1, .Llarge # 如果 (fact > 100) 跳转到 large
47
48     la      a0, .str.4
49     call    printf
50     j       .Lendif
51
52 .Llarge:
53     la      a0, .str.3
54     call    printf
55
56 .Lendif:
57     # for (i = 0; i < 5; i++)
58     sw      zero, 20(sp)  # i = 0
59
60 .Lloop_cond:
61     lw      t0, 20(sp)    # t0 = i
62     li      t1, 5         # t1 = 5
63     bge     t0, t1, .Lloop_end # 如果 (i >= 5) 跳转到 loop_end
64
65     la      a0, .str.5
66     lw      a1, 20(sp)    # a1 = i
67     call    printf
68

```

```

69     # i++
70     lw     t0, 20(sp)      # t0 = i
71     addi   t0, t0, 1      # t0 = i + 1
72     sw     t0, 20(sp)      # i = t0
73     j      .Lloop_cond    # 继续循环
74
75 .Lloop_end:
76     # 初始化数组 arr[5] = {1, 2, 3, 4, 5}
77     li     t0, 1
78     sw     t0, 24(sp)      # arr[0] = 1
79     li     t0, 2
80     sw     t0, 28(sp)
81     li     t0, 3
82     sw     t0, 32(sp)
83     li     t0, 4
84     sw     t0, 36(sp)
85     li     t0, 5
86     sw     t0, 40(sp)
87
88     la     a0, .str.6
89     lw     a1, 32(sp)      # a1 = arr[2]
90     call   printf
91
92     # return 0
93     li     a0, 0          # 返回值 = 0
94
95     # 恢复寄存器并返回
96     ld     ra, 72(sp)
97     ld     s0, 64(sp)
98     ld     s1, 56(sp)
99     ld     s2, 48(sp)
100    addi   sp, sp, 80
101    ret

```

4. 汇编程序验证

编写完源代码的汇编程序后，我们使用指令

```

1 riscv64-unknown-elf-as -march=rv64gc -mabi=lp64d main.s -o main.o
2 riscv64-unknown-elf-gcc -c ../../lib/sylib.c -march=rv64gc -mabi=lp64d -o ../../lib/sylib.o
3 riscv64-unknown-elf-gcc main.o ../../lib/sylib.o -o main\_riscv

```

生成可执行文件并使用 qemu 运行可执行文件得到如下输出：

```

docay@DESKTOP-R79JUA1:~/Compiler-Principles/lab1/t2/rics-v$ qemu-riscv64 main_riscv
Sum of 5 and 10 is 15
Factorial of 5 is 120
Fibonacci of 6 is 8
The factorial is large
Loop iteration 0
Loop iteration 1
Loop iteration 2
Loop iteration 3
Loop iteration 4
Array element at index 2: 3
TOTAL: 0H-0M-0S-0us

```

图 3: 汇编验证

从上述输出中可以看出，我们编写的汇编程序与 SysY 源程序等价、能链接 SysY 运行库并且能得到正确的结果。

5. 汇编代码比较

上面我们已经编写完正确的汇编代码，我们在这一部分主要探究我们编写的汇编代码和编译器生成的汇编代码之间的差异，我们给出编译器生成的部分汇编代码：

```

1  add:
2      addi    sp,sp,-32
3      sd      s0,24(sp)
4      addi    s0,sp,32
5      mv      a5,a0
6      mv      a4,a1
7      sw      a5,-20(s0)
8      mv      a5,a4
9      sw      a5,-24(s0)
10     lw      a5,-20(s0)
11     mv      a4,a5
12     lw      a5,-24(s0)
13     addw     a5,a4,a5
14     sext.w   a5,a5
15     mv      a0,a5
16     ld      s0,24(sp)
17     addi    sp,sp,32
18     jr      ra
19     .size   add, .-add
20     .align  1
21     .globl  factorial
22     .type   factorial, @function
23 factorial:
24     addi    sp,sp,-32
25     sd      ra,24(sp)
26     sd      s0,16(sp)
27     addi    s0,sp,32
28     mv      a5,a0

```

```
29     sw     a5,-20(s0)
30     lw     a5,-20(s0)
31     sext.w  a4,a5
32     li     a5,1
33     bgt     a4,a5,.L10
34     li     a5,1
35     j      .L11
36 .L10:
37     lw     a5,-20(s0)
38     addiw   a5,a5,-1
39     sext.w  a5,a5
40     mv     a0,a5
41     call    factorial
42     mv     a5,a0
43     lw     a4,-20(s0)
44     mulw    a5,a4,a5
45     sext.w  a5,a5
46 .L11:
47     mv     a0,a5
48     ld     ra,24(sp)
49     ld     s0,16(sp)
50     addi    sp,sp,32
51     jr     ra
52     .size   factorial,.-factorial
53     .align  1
54     .globl  fibonacci
55     .type   fibonacci, @function
56 fibonacci:
57     addi    sp,sp,-48
58     sd     ra,40(sp)
59     sd     s0,32(sp)
60     sd     s1,24(sp)
61     addi    s0,sp,48
62     mv     a5,a0
63     sw     a5,-36(s0)
64     lw     a5,-36(s0)
65     sext.w  a4,a5
66     li     a5,1
67     bgt     a4,a5,.L13
68     lw     a5,-36(s0)
69     j      .L14
70 .L13:
71     lw     a5,-36(s0)
72     addiw   a5,a5,-1
73     sext.w  a5,a5
74     mv     a0,a5
75     call    fibonacci
76     mv     a5,a0
77     mv     s1,a5
```

```

78     lw     a5,-36(s0)
79     addiw  a5,a5,-2
80     sext.w  a5,a5
81     mv     a0,a5
82     call   fibonacci
83     mv     a5,a0
84     addw   a5,s1,a5
85     sext.w  a5,a5
86 .L14:
87     mv     a0,a5
88     ld     ra,40(sp)
89     ld     s0,32(sp)
90     ld     s1,24(sp)
91     addi   sp,sp,48
92     jr     ra
93     .size   fibonacci,.-fibonacci
94     .ident  "GCC: (13.2.0-11ubuntu1+12) 13.2.0"

```

随后我们在终端中使用 `diff` 指令比较编写的汇编代码和生成汇编代码之间的区别，我们可以将区别总结为以下几点：

1. 文件头和声明：编写的汇编代码中直接使用 `.section .rodata`、`.section .text`，用 `.str`、`.str.1`... 来保存字符串。而编译器生成的代码则自动加入了 `.file`、`.option nopic`、`.attribute arch` 等元信息，并且将字符串常量命名为 `.LC1`、`.LC2`...，每个常量之间带有 `.align 3`，还额外生成 `.LC0` 用于存放数组。
2. `add` 函数：手写版本非常简洁，仅用一条 `add` 指令和 `ret` 返回即可。而编译器版本会建立栈帧，保存和恢复寄存器，并将参数先写入栈再读出进行加法，逻辑等价但更冗长。
3. `factorial` 函数：两者逻辑一致，都是通过递归实现 $n!$ 的计算。手写代码在递归和条件判断上更简洁；编译器代码额外使用了 `sext.w`、`mulw` 等指令来保证 32 位有符号运算，同时寄存器保存/恢复更完整。
4. `fibonacci` 函数：手写版本是标准的递归写法，通过 `fibonacci(n-1)+fibonacci(n-2)` 实现。编译器版本逻辑相同，但使用 `addw`、`sext.w` 保证 32 位扩展，并且借助寄存器保存中间结果，显得更机械化。
5. `main` 函数：手写版本显式分配和管理局部变量，使用 `printf` 打印结果，并通过 `li+sw` 初始化数组。编译器版本则使用 `putf`，并将数组提前存放在数据段（`.LC0`）中再加载；同时其栈帧建立和条件跳转更规范化。
6. 元信息：手写版本直接以 `ret` 返回结束，而编译器生成的代码会带有 `.size`、`.ident "GCC: ..."` 等 ELF 文件格式相关的元数据。

四、 实验总结

本次实验通过深入探究编译系统的完整工作流程，系统地研究了从源代码到可执行文件的各个编译阶段，并成功实现了从 SysY 到 LLVM IR 再到 RISC-V 汇编的完整编译链路。

在编译流程探究方面，我们以一个包含递归函数调用的 Fibonacci 程序为例，详细分析了 GCC/LLVM 工具链在预处理、编译、汇编和链接各阶段的具体工作机制。通过实验验证，我们观察到预处理器如何展开宏定义、处理文件包含和删除注释；编译器如何进行词法分析生成 token 流、语法分析构建 AST、语义分析检查类型安全，以及生成中间代码 LLVM IR；汇编器如何将汇编代码转换为机器码目标文件；链接器如何解决符号引用并生成最终可执行文件。特别是通过对比不同优化级别（O0-O3）下的代码生成结果，我们发现 O2 优化在性能提升和编译开销之间取得了较好的平衡。

在 SysY 程序实现方面，我们设计了一个涵盖编译器需支持的核心语言特性的示例程序，包括函数声明与调用、递归、条件分支、循环、数组操作等。基于此，我们手工编写了等价的 LLVM IR 程序。通过对比不同优化级别下 LLVM IR 的变化，我们观察到了常量折叠、尾递归优化、控制流简化等编译优化技术的实际效果。此外，我们还尝试了 Polly 自动并行化优化，发现其在高级优化级别下才能有效识别可并行化的循环结构。

在 RISC-V 汇编实现方面，我们手工编写了与 SysY 源程序等价的 RISC-V 汇编代码，实现了加法、阶乘递归、Fibonacci 递归等函数，以及包含变量初始化、函数调用、条件分支、循环和数组操作的主函数。通过与编译器自动生成的汇编代码对比，我们发现手写版本在代码简洁性上有优势，而编译器生成版本则在栈帧管理、寄存器保存恢复、符号扩展等方面更加规范和安全。最终，我们成功将汇编程序与 SysY 运行库链接，并在 QEMU 模拟器上验证了运行结果的正确性。

通过本次实验，我们不仅掌握了编译系统各阶段的工作原理和实现细节，还深入理解了中间表示设计、代码优化策略以及目标代码生成的关键技术，为后续深入学习编译原理和开发实际编译器奠定了坚实基础。